

Kapitelübersicht

- 1 Einführung
- 2 Objektorientierung
- 3 Vertiefung C++**
- 4 Die Standard Template Library (STL)
- 5 Clean Code
- 6 Test-driven Development

Recap

RAII

`std::vector`

Compiler Fehler und Intellisense

Kapitel 3: Vertiefung C++



31. Weitere C++ Keywords

32. Namespaces

33. Objektorientiertes Design

34. Übungsprojekt

const

- Zur Definition von Variablen die nicht mehr verändert werden sollen (z.B. Konstanten)
- Keyword gibt an, dass die jeweilige Variable zur Laufzeit nicht mehr geändert werden kann

const

- Zur Definition von Variablen die nicht mehr verändert werden sollen (z.B. Konstanten)
- Keyword gibt an, dass die jeweilige Variable zur Laufzeit nicht mehr geändert werden kann

```
#include <iostream>
#include <cmath>

const double MAGNUS_BASE_HPA = 6.112;
const double MAGNUS_COEFF_A = 17.67;
const double MAGNUS_COEFF_B_C = 243.5;
const double HUMIDITY_CONVERSION_FACTOR = 2.1674;
const double KELVIN_OFFSET = 273.15;

struct SensorData
{
    double temperature;
    double relativeHumidity;
};

void getSensorInputs(SensorData &sensorData)
{
    std::cout << "Temperature in °C: ";
    std::cin >> sensorData.temperature;

    std::cout << "Relative Humidity in %: ";
    std::cin >> sensorData.relativeHumidity;
};

double calculateAbsoluteHumidity(const SensorData &sensorData)
{
    const double numerator = MAGNUS_BASE_HPA * std::exp((MAGNUS_COEFF_A * sensorData.temperature) / (MAGNUS_COEFF_B_C + sensorData.temperature));
    const double denominator = KELVIN_OFFSET + sensorData.temperature;
    const double absoluteHumidity = numerator / denominator;
    return absoluteHumidity;
};

int main()
{
    SensorData sensorData;

    getSensorInputs(sensorData);

    const double absoluteHumidity = calculateAbsoluteHumidity(sensorData);
    std::cout << "Absolute Humidity: " << absoluteHumidity << " g/m³" << std::endl;

    return 0;
}
```

Berechnung der absoluten Luftfeuchtigkeit

Dieses Programm berechnet die absolute Luftfeuchtigkeit in g/m³ basierend auf Temperatur und relativer Luftfeuchtigkeit.

Formel

$$\rho_{abs} = \frac{6,112 \cdot \exp\left(\frac{17,67 \cdot T}{T + 243,5}\right) \cdot r_H \cdot 2,1674}{273,15 + T}$$

Dabei:

- T : Temperatur in °C
- r_H : Relative Luftfeuchtigkeit in %
- ρ_{abs} : Absolute Luftfeuchtigkeit in g/m³

Konstanten in der Formel

- **6,112**: Sättigungsdampfdruck bei 0 °C (Magnus-Formel, Basiswert in hPa)
- **17,67**: Magnus-Koeffizient A (empirischer Wert für Wasser)
- **243,5**: Magnus-Koeffizient B in °C (empirischer Wert für Wasser)
- **2,1674**: Umrechnungsfaktor aus idealer Gasgleichung (für Einheit g/m³)
- **273,15**: Kelvin-Offset (Umrechnung von °C nach Kelvin)

Hinweise zur Implementierung

- **const-Konstanten**: Alle Zahlenwerte sind als benannte Konstanten definiert
- **SensorData struct**: Temperatur und relative Luftfeuchtigkeit in einer Struktur
- **const Referenzen**: Vermeidung unnötigen Kopierens

const

- OOP: Methoden können `const` sein → Sie dürfen keine Member ändern
- OOP: Membervariablen (Argumente) können `const` sein → Diese dürfen von der Funktion/Methode nicht geändert werden
- `const`-Membervariablen müssen bei Objekterzeugung initialisiert werden (Initialisierungsliste des Konstruktors oder In-Class-Initialisierer im Header)

const

- OOP: Methoden können `const` sein → Sie dürfen keine Member ändern
- OOP: Membervariablen (Argumente) können `const` sein → Diese dürfen von der Funktion/Methode nicht geändert werden

The screenshot shows a code editor with the following C++ code for `HumidityCalculator.hpp`:

```
1 #ifndef HUMIDITY_CALCULATOR_HPP
2 #define HUMIDITY_CALCULATOR_HPP
3
4 #include <iostream>
5 #include <cmath>
6
7 class HumidityCalculator
8 {
9 public:
10     HumidityCalculator(const double temp, const double humidity);
11
12     double calculateAbsoluteHumidity() const;
13
14     double getTemperature() const;
15
16     double getRelativeHumidity() const;
17
18 private:
19     const double m_temperatureCelsius;
20     const double m_relativeHumidity;
21     constexpr static double CONSTANT_FACTOR = 6.112 * 2.167;
22
23 };
24
25 #endif // HUMIDITY_CALCULATOR_HPP
26
27 // constexpr berechnet den Wert zur Compile-Zeit, was die Effizienz erhöht.
28 // zur Laufzeit liegt der Wert vor 13.222064 (6.112 * 2.167).
29
30 // Welche Werte sollten als const, welche als constexpr deklariert werden?
31
```

The sidebar contains the following text:

Erweiterung: Verwendung von const in Methoden und Member-Variablen

Um die Verwendung von `const` in Methoden und Member-Variablen zu demonstrieren, wurde das Beispiel erweitert. Die neue Implementierung verwendet eine Klasse `HumidityCalculator`, die die absolute Luftfeuchtigkeit berechnet.

Neue Klasse: HumidityCalculator

Die Klasse `HumidityCalculator` enthält:

- **const Member-Variablen:** `temperatureCelsius` und `relativeHumidity` sind konstant und können nach der Initialisierung nicht geändert werden.
- **const Methoden:** Methoden wie `calculateAbsoluteHumidity`, `getTemperature`, und `getRelativeHumidity` sind konstant und ändern keine Member-Variablen.
- **Konstruktor mit const Parametern:** Der Konstruktor stellt sicher, dass die übergebenen Werte nicht innerhalb des Konstruktors geändert werden.

Vorteile der Erweiterung

- **Sicherheit:** `const` stellt sicher, dass Member-Variablen und Methoden keine unerwarteten Änderungen vornehmen.
- **Lesbarkeit:** Der Code ist klarer und zeigt die Absicht, dass bestimmte Werte unveränderlich sind.
- **Optimierung:** Der Compiler kann Optimierungen vornehmen, da er weiß, dass bestimmte Werte konstant sind.

Erweiterung: Verwendung von constexpr

Um die Effizienz und Sicherheit des Codes zu erhöhen, wurde die Verwendung von `constexpr` ergänzt. `constexpr` ermöglicht die Berechnung von Werten zur Compile-Zeit, wodurch Laufzeitkosten reduziert werden.

Vorteile von constexpr (Compile-Time Constants)

- **Compile-Zeit-Berechnung:** Werte, die mit `constexpr` deklariert sind, werden bereits zur Compile-Zeit berechnet, was die Effizienz erhöht.
- **Konstante Werte:** `constexpr` stellt sicher, dass die Werte unveränderlich sind und zur Laufzeit nicht geändert werden können.

`const`-Variablen dürfen nicht nachträglich gesetzt werden.
Sie müssen sofort beim Erzeugen (entweder im Header oder in der Initialisierungsliste) einen Wert bekommen.

constexpr

- Compile-Zeit-Auswertung → Wert/Berechnung wird bereits beim Kompilieren festgelegt
- Performance-Vorteil → Keine Berechnung zur Laufzeit nötig
- Implizit const → Wert ist unveränderlich

Wann sollten Sie `const` / `constexpr` verwenden?

Wann immer möglich!

Aufgabe: const

- Verwenden Sie das die Aufgabe constEdit
(3_Basics_und_Vertiefungen_Cpp\31_Vertiefung\311_Keywords\tasks\constEdit\constEdit.cpp)
- Setzen Sie alles `const` was `const` gesetzt werden kann (und auch sollte).
- Versuchen Sie nicht zu raten sondern zu wissen.

static

- Alle bisher genutzten Methoden/Variablen existierten auf Objektebene
- Zum Aufruf der Methoden muss vorher ein Objekt erzeugt sein
- Die Werte der Variablen können in jedem Objekt dieser Klasse unterschiedlich sein
- Mithilfe von `static` lassen sich **Variablen/Methoden definieren, die objektlos aufgerufen werden können**
- Dieser Wert/Methode ist dann in allen Objekten dieser Klasse gleich
- Die Methode hat nie Abhängigkeiten zu den Daten eines spezifischen Objekts

static

- Alle bisher genutzten Methoden/Variablen existierten auf Objektebene
- Zum Aufruf der Methoden muss vorher ein Objekt erzeugt sein
- Die Werte der Variablen können in jedem Objekt dieser Klasse unterschiedlich sein
- Mithilfe von `static` lassen sich **Variablen/Methoden definieren, die objektlos aufgerufen werden können**
- Dieser Wert/Methode ist dann in allen Objekten dieser Klasse gleich
- Die Methode hat nie Abhängigkeiten zu den Daten eines spezifischen Objekts

- Mithilfe von `static` lassen sich **Variablen/Methoden** definieren, die **objektlos aufgerufen werden können**

```
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36
1 #ifndef HUMIDITY_CALCULATOR_HPP
2 #define HUMIDITY_CALCULATOR_HPP
3
4 #include <iostream>
5 #include <cmath>
6
7 class HumidityCalculator
8 {
9 public:
10     HumidityCalculator(const double temp, const double humidity);
11
12     virtual ~HumidityCalculator()
13     {
14         std::cout << "HumidityCalculator destructor called.\n";
15     }
16
17     double calculateAbsoluteHumidity() const;
18
19     double getTemperature() const;
20
21     double getRelativeHumidity() const;
22
23     static int getInstanceCount()
24     {
25         return m_instanceCount;
26     }
27
28 private:
29     const double m_temperatureCelsius;
30     const double m_relativeHumidity;
31     static constexpr double M_CONSTANT_FACTOR{6.112 * 2.167};
32     static int m_instanceCount;
33 };
34
35 #endif // !HUMIDITY_CALCULATOR_HPP
```

03_static - Implementierung: Verwendung von static in Member-Variablen

Die Implementierung `03_static` demonstriert die Verwendung von `static` Member-Variablen in der Klasse `HumidityCalculator`. Dies ermöglicht die Verfolgung der Anzahl der zur Laufzeit erstellten Instanzen.

Hauptmerkmale

- **Statische Member-Variablen:** `m_instanceCount` ist eine statische Variable, die die Gesamtzahl der Instanzen der Klasse `HumidityCalculator` verfolgt.
- **Instanzzählung:** Die Methode `getInstanceCount` bietet Zugriff auf die aktuelle Anzahl der Instanzen.
- **Destruktor-Protokollierung:** Der Destruktor protokolliert, wann eine Instanz zerstört wird, und bietet Einblicke in das Management des Objektlebenszyklus.

Die Implementierung `03_static` zeigt, wie `static` Variablen effektiv verwendet werden können, um klassenbezogene Daten zu verwalten und zu verfolgen, wodurch sowohl die Funktionalität als auch die Effizienz verbessert werden.

Zusammenfassung des Keywords-Projekts

Vorteile der Verwendung von `const` in Methoden und Member-Variablen

- **Sicherheit:** `const` stellt sicher, dass Member-Variablen nicht versehentlich geändert werden.
- **Sicherheit:** `const` stellt sicher, dass Methoden keine unerwarteten Änderungen vornehmen.
- **Lesbarkeit:** Der Code ist klarer und zeigt die Absicht, dass bestimmte Werte unveränderlich sind.
- **Optimierung:** Der Compiler kann Optimierungen vornehmen, da er weiß, dass bestimmte Werte konstant sind.

Vorteile von `constexpr` (Compile-Time Constants)

- **Compile-Zeit-Berechnung:** Werte, die mit `constexpr` deklariert sind, werden bereits zur Compile-Zeit berechnet, was die Effizienz erhöht.
- **Konstante Werte:** `constexpr` stellt sicher, dass die Werte unveränderlich sind und zur Laufzeit nicht geändert werden können.
- **Optimierung:** Der Compiler kann Optimierungen vornehmen, da er weiß, dass bestimmte Werte konstant sind.

Aufgabe: static

Verwenden Sie die Aufgabenbeschreibung aus dem Git Repo:

3_Basics_und_Vertiefungen_Cpp\31_Vertiefung\311_Keywords\tasks\task_user_static_id\TASK_USER_STATIC_ID.md

auto

- Kann anstelle des konkreten Typs bei einer Zuweisung angegeben werden, wenn direkt ein Wert zugewiesen wird
- Compiler ermittelt selbst den Typ
- Typsicherheit bleibt erhalten, d.h. Compiler warnt weiter wenn Typen nicht zusammen passen
- C++ Standard: AAA: Almost Always Auto
- Aber: Gefahr Lesbarkeit zu reduzieren!
- Besser: Auto nur wenn Typ auf rechten Seite klar definiert
- Auto nur wenn Lesbarkeit erhöht wird, z.B. bei zu langen Klassennamen

Welche Verwendung von auto ist sinnvoll?

```
auto numberOfUsers{5};  
int measValue{5.5};  
...  
auto humCalculator{HumidityCalculator(20, 50)};  
auto humCalculator = HumidityCalculator(20, 50);  
...  
auto measValue02{measValue}
```

Friend class

- Variablen, die in der Regel `private` sein sollen, werden **von einer einzelnen spezifischen Klassen** benötigt
- hierzu wäre es nicht sinnvoll diese komplett als `public` zu definieren
- Die andere Klasse kann als `friend` deklariert werden
- `friend` Beziehungen gelten nur in eine Richtung
- Die Klasse `FriendClass` bekommt Zugriff auf die `private` Variablen der Klasse `Class`
- **Z.B. in Tests sinnvoll**

```
class Class
{
public:
    Class()
    : m_privateMember(5)
    {};

    ~Class()
    {};

    friend class FriendClass; //define FriendClass to be friend

private:
    int m_privateMember; //private member variable
};

class FriendClass
{
public:
    void testAccess(const Class& testObj)
    {
        std::cout << testObj.m_privateMember << std::endl; //has access to private member of Class
    }
};
```


Aufgabe: keywords

Verwenden Sie die Aufgabenbeschreibungen aus dem Git Repo:

- **3_Basics_und_Vertiefungen_Cpp\31_Vertiefung\311_Keywords\tasks\BankAccounts\TASK_BANKACCOUNTS.md**

Kapitel 3: Vertiefung C++

31. Weitere C++ Keywords



32. Namespaces

33. Objektorientiertes Design

34. Übungsprojekt

Namespaces - Motivation

- Annahme:
Wir implementieren ein Algebra Programm und wollen darin eine Klasse `Vector` implementieren
- Problem: `Vector` gibt es schon! → Compiler Fehler
- Lösung: Namespaces! `Vector` der C++ Standard Template Library (STL) liegt im Namespace `std` und wird aufgerufen mit `std::vector` → Nur `std::vector` ist belegt, wir können unsere Klasse `Vector` nennen.
- Für große Projekte und oft wieder verwendeten Code unverzichtbar

Vorteile von Namespaces:
Verhindern von Namenskollisionen
Verbesserte Lesbarkeit (z.B. `algebra::vector`)

Namespaces - Beispiel

- Bei großen Projekten: ALLES in Namespaces mit Namenskonventionen für die Namespaces, z.B. jedes Projekt einen eigenen Namespace
- Meine Empfehlung: In Namespaces nicht einrücken, da sonst oft ganze Files einfach ein oder zweimal eingerückt werden
→ weniger Platz zum Vergleichen
- Aber Ende des Namespaces kommentieren, wofür die geschlossene Klammer steht

```
1  #include <vector>
2
3
4  namespace Algebra
5  {
6
7      class Vector
8      {
9
10     };
11
12 } // end namespace algebra
13
14
15 int main()
16 {
17     // create STL vector
18     std::vector<int> vec;
19     // create algebra vector
20     Algebra::Vector vec;
21
22     return 0;
23 }
```

Erster Buchstabe groß. Nach includes, vor class

Im Namespace nicht eingerückt

Ende kommentiert

Namespaces – .hpp and .cpp files

```
3_Vertiefung_Cpp > namespaces > C++ main.cpp > main()
1  #include <vector>
2
3  #include "classInNamespace.hpp"
4
5
6  int main()
7  {
8
9      MyNamespace::ClassInNamespace myObject;
10
11      using MyNamespace::ClassInNamespace;
12
13      ClassInNamespace myObject;
14
15      return 0;
16  }
17
18

3_Vertiefung_Cpp > namespaces > h++ classInNamespace.hpp > ...
1
2  namespace MyNamespace
3  {
4
5      class ClassInNamespace
6      {
7
8      };
9
10 } // end namespace MyNamespace

3_Vertiefung_Cpp > namespaces > C++ classInNamespace.cpp > {} MyNamespace
1  #include "classInNamespace.hpp"
2
3  namespace MyNamespace
4  {
5
6      ClassInNamespace::ClassInNamespace()
7      {
8          // Implementation of Default Constructor
9      }
10
11 } // end namespace MyNamespace
```

- Namespaces können sich über mehrere Dateien erstrecken
- Wenn zusätzliche Klassen und Funktionen in einen Namespace eingefügt werden sollen oder die Implementierungen in der .cpp Datei, dann geht dies einfach über das erneute definieren des Namespaces.

using namespace

- Ganze Namespaces können über `using` in den aktuellen Namespace eingebunden werden (z.B. `using namespace std;`)

using namespace: Nur mit viel Vorsicht!
Und niemals in Headern!

- Auch Vorsicht vor `using namespace std;`
 - STL wird häufig genutzt: erscheint sinnvoll
 - Aber STL ist im stetigen Wandel
 - Namenskollisionen können entstehen obwohl eigener Code nicht geändert wurde
- Alternative: einzelne oft genutzte Ausdrücke aus `namespace` befreien durch z.B. `using std::cout;` (geht auch mit eigenen langen Namespaces)